# 01 TS Starter

# TypeScript: A Comprehensive Introduction & Setup Guide

## What is TypeScript?

TypeScript is **JavaScript with syntax for types**. It is a strongly typed programming language that builds on JavaScript, providing better tooling at any scale. Fundamentally, TypeScript serves as a developer's tool that helps write better, more maintainable JavaScript code.

### Key Characteristics

- **Superset of JavaScript**: TypeScript extends JavaScript, meaning all valid JavaScript code is valid TypeScript code
- **Strongly Typed**: Unlike JavaScript's dynamic typing, TypeScript enforces type safety at compile time
- **Compiles to JavaScript**: TypeScript code is compiled (transpiled) to standard JavaScript that can run in any browser or JavaScript environment
- **Created by Microsoft**: Developed by Anders Hejsberg, who also created C#
- **Developer Tool**: TypeScript doesn't run in browsers; it's a development-time tool that produces JavaScript

### Why TypeScript?

TypeScript provides enhanced tooling and catches errors before runtime. According to the Stack Overflow Developer Survey, TypeScript consistently ranks among the top programming languages, demonstrating its widespread adoption and value in modern web development.

## Prerequisites

Before working with TypeScript, you should have:

- **Solid JavaScript fundamentals**: Understanding of variables, functions, data types, and basic JavaScript concepts
- **Familiarity with modern JavaScript**: ES6+ features like let/const, arrow functions, etc.

## Environment Setup

### Required Software

1. **Visual Studio Code** (recommended code editor)
   - Download from: code.visualstudio.com
   - Excellent TypeScript integration built-in
   - Available for Windows, macOS, and Linux
2. **Node.js and npm**

- Download the LTS (Long Term Support) version from nodejs.org
- Provides npm (Node Package Manager), which is essential for installing TypeScript
- Not required for Node.js development specifically, but for the package manager

### Installing TypeScript Globally

Open a terminal and run:

```shell
npm install typescript -g
```

The `-g` flag installs TypeScript globally, making it available across all projects on your system.

## Creating Your First TypeScript Project

### Basic Project Structure (Single File)

1. **Create an HTML file** (`index.html`):

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TypeScript Project</title>
    <style>
        body {
            background-color: black;
        }
    </style>
</head>
<body>
    <script src="main.js" defer></script>
</body>
</html>
```

2. **Create a TypeScript file** (`main.ts`):

```ts
let username = "Kirk";
console.log(username);
```

3. **Compile TypeScript to JavaScript**:

```shell
tsc main.ts
```

This creates `main.js`, which is the compiled JavaScript file.

4. **Watch mode for continuous compilation**:

```shell
tsc main.ts -w
```

The `-w` (watch) flag monitors the file for changes and automatically recompiles when you save.

### Important Notes on Compilation

- TypeScript compiles to JavaScript compatible with older browsers by default
- For example, `let` in TypeScript might become `var` in compiled JavaScript (depending on target configuration)
- The compiled JavaScript may look different from your TypeScript, but maintains the same functionality

## Professional Project Structure

For larger, scalable projects, organize your files properly:

```
project-root/
├── src/              # Source TypeScript files
│   └── main.ts
├── build/            # Compiled output
│   ├── index.html
│   ├── css/
│   └── js/           # Compiled JavaScript files
│       └── main.js
└── tsconfig.json     # TypeScript configuration
```

### Setting Up the Configuration File

1. **Initialize TypeScript configuration**:

```shell
tsc --init
```

This creates a `tsconfig.json` file with default settings and extensive comments.

2. **Essential Configuration Options**:

Open `tsconfig.json` and modify these key settings:

```
{
  "compilerOptions": {
    "target": "ES2016",                    // JavaScript version to compile to
    "rootDir": "./src",                    // Where TypeScript files are located
    "outDir": "./build/js",                // Where compiled JS files go
    "noEmitOnError": false                 // Whether to compile despite errors,
set true in prod
  },
  "include": ["src"]                       // Only compile files in src directory
}
```

## Configuration Breakdown

`target`: Specifies which JavaScript version to compile to

- `ES5`: Compatible with older browsers (uses `var`)
- `ES2016`: Modern JavaScript (uses `let`/`const`)
- Higher versions: Support more modern features

`rootDir`: Source directory for TypeScript files

- Set to `"./src"` to keep source code organized
- TypeScript will only look here for `.ts` files

`outDir`: Destination for compiled JavaScript

- Set to `"./build/js"` to separate compiled code from source
- Creates nested directory structure automatically

`noEmitOnError`: Controls compilation behavior

- `false` (default): Compiles even with TypeScript errors
- `true`: Prevents compilation if any errors exist

`include`: Array of directories/patterns to include

- Prevents TypeScript from compiling files outside specified directories
- Example: `["src"]` only compiles files in the src folder

## Running TypeScript with Configuration

Once `tsconfig.json` is configured:

```shell
tsc -w
```

This command:

- Uses settings from `tsconfig.json`
- Watches all files in the specified `rootDir`
- Compiles to the specified `outDir`
- No need to specify individual files

### Alternative: Command-Line Flags

You can override configuration file settings using command-line flags:

```shell
tsc --noEmitOnError -w
```

This enforces the no-emit-on-error rule regardless of the configuration file setting.

## Understanding Type Safety

### Type Inference

TypeScript automatically infers (figures out) types based on assigned values:

```
let a = 12;         // TypeScript infers: number
let b = "6";        // TypeScript infers: string
let c = 2;          // TypeScript infers: number
```

### Type Errors and Warnings

Consider this code:

```
let a = 12;
let b = "6";
let c = 2;

console.log(a / b);  // TypeScript error: arithmetic operation on string
console.log(c * b);  // TypeScript error: arithmetic operation on string
```

**What happens:**

- JavaScript allows this (type coercion) and would output `2` and `12`
- TypeScript shows red squiggly lines warning about type mismatches
- The code still compiles to JavaScript by default
- The compiled JavaScript will run, but TypeScript warns you it's problematic

**Error message:**

> "The right-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type."

### Explicit Type Annotations

To write better TypeScript, explicitly declare types:

```
let a: number = 12;
let b: number = 6;
let c: number = 2;

console.log(a / b);  // No error: 2
console.log(c * b);  // No error: 12
```

This approach:

- Makes your intentions clear
- Catches type errors immediately
- Prevents accidental type mismatches
- Improves code readability and maintainability

## Best Practices

### 1. Use Explicit Types When Appropriate

While TypeScript can infer types, being explicit helps prevent errors and documents your code.

### 2. Enable Strict Checking

Consider setting `noEmitOnError: true` in production environments to prevent compilation when errors exist.

### 3. Organize Files Properly

- Keep source TypeScript in a dedicated directory
- Compile to a separate build directory
- Never edit compiled JavaScript files directly

### 4. Remove Unused Files

When deleting a TypeScript file, remember to manually delete the corresponding compiled JavaScript file (TypeScript doesn't do this automatically).

### 5. Update HTML References

When restructuring projects, update script tags to reflect new file locations:

HTML

```
<script src="js/main.js" defer></script>
```

## Key Takeaways

1. **Valid JavaScript is valid TypeScript** – You can use existing JavaScript code in TypeScript files

2. **TypeScript compiles to JavaScript** – Browsers run the compiled JavaScript, not TypeScript
3. **Type safety is a development tool** – It catches errors before runtime
4. **The compiler provides warnings** – Even when code compiles, warnings indicate potential issues
5. **Configuration is powerful** – `tsconfig.json` controls how TypeScript behaves
6. **Strong typing helps long-term** – Initial overhead pays off in maintainability and error prevention

## Common Issues and Solutions

### Issue: "Cannot redeclare block-scoped variable"

**Cause**: Same variable name exists in both TypeScript and compiled JavaScript files when both are open

**Solution**: Close the JavaScript file; you should only edit TypeScript files

### Issue: TypeScript compiles files outside src directory

**Cause**: Missing or incorrect `include` configuration

**Solution**: Add `"include": ["src"]` to `tsconfig.json`

### Issue: Compiled JavaScript doesn't update

**Cause**: TypeScript isn't watching for changes

**Solution**: Run `tsc -w` or restart the watch process

### Issue: Changes not reflected in browser

**Cause**: Old JavaScript cached or not recompiled

**Solution**:

- Ensure TypeScript watch mode is running
- Hard refresh browser (Ctrl+Shift+R or Cmd+Shift+R)
- Check that HTML links to correct JavaScript file

## Conclusion

TypeScript enhances JavaScript development by adding static type checking, better tooling, and improved code quality. While it requires compilation and adds some overhead to the development process, the benefits of catching errors early, improving code documentation, and enabling better IDE support make it invaluable for modern JavaScript development.

The key is to embrace the type system rather than fight it – TypeScript's warnings are designed to help you write better code.